

POLITECNICO DI TORINO



Lab 4

Exercise 3 - Web

Music and multimedia streaming over the Internet

Academic year 2025/2026

Full Name : MEHRAN BAYAT

Student Number : S320636.

Cristina Rottondi, Leonardo Severi, Massimiliano Zanoni

Version: 2026

©2025-26

README

Lab 4 Exercise 3: Web

HTTP and HTTPS Login Traffic Analysis Using Wireshark
Loopback Interface, Localhost Browser Test, and TLS Encryption

Final_Report.md

Contents

1 Title	3
2 Abstract	3
3 Exercise Objective	3
4 Lab Environment and Tools	4
4.1 Working Environment	4
4.2 Wireshark Capture Interface	4
4.3 Files Used in the Experiment	4
5 Credentials Used for the Test	4
6 HTTP Server Experiment	5
6.1 Server Execution	5
6.2 Packets Observed in Wireshark	5
6.3 Username and Password Visibility in HTTP	6
6.4 Interpretation of the HTTP Result	6
7 HTTPS Server Experiment	6
7.1 Server Execution	7
7.2 Packets Observed in Wireshark	8
7.3 HTTP Filter Result in the HTTPS Capture	9
7.4 Username and Password Visibility in HTTPS	9
8 HTTP and HTTPS Comparison	10
8.1 Main Difference	10
8.2 Security Interpretation	10
9 Direct Answers for the Lab Deliverable	10
9.1 4.3.1 HTTP Server	10
9.2 4.3.2 HTTPS Server	11
10 Final Summary	11
References	12

1. Title

HTTP and HTTPS Login Traffic Analysis Using Firefox, Terminal, and Wireshark in the Crownlabs Lab 2 Virtual Machine

2. Abstract

In this report, I explain what I did for **Lab 4 - Exercise 3: Web**. In this exercise, I compared the traffic of a simple login page in two cases: first with **HTTP** and then with **HTTPS**. I used the same browser, the same local server address, and the same login values in both tests, so the difference between the two captures could be seen clearly in Wireshark.

For the HTTP part, I started the server with `python3 test_server.py`. Then I opened `http://localhost:8080` in Firefox, typed the username `mehran` and the password `test123`, and checked the packet generated after pressing the login button. In Wireshark, the `POST /login` request showed the login data directly in the body:

```
{"user": "mehran", "pwd": "test123"}
```

For the HTTPS part, I repeated the same procedure with `python3 test_server_https.py` and opened `https://localhost:8080`. Firefox showed a security warning because the local server certificate was self-signed, so I clicked the advanced option and accepted the risk to continue. The terminal of the server still printed the username and password, but Wireshark did not show them as readable text. This is the main point of this exercise: the HTTPS server can decrypt the data, but the packet capture only sees encrypted traffic.

The final result was very clear for me: **with HTTP, the username and password were visible in clear text**, while **with HTTPS, I could not read them from the packets**. This showed practically why HTTPS is needed when a web page sends login information.

3. Exercise Objective

The goal of this Web exercise was to see with my own capture how web traffic appears in Wireshark and how encryption changes what I can read from the packets.

The exercise had two required parts:

1. **HTTP Server:** run the plain HTTP server, open the login page, submit credentials, and check if the username and password can be found in Wireshark.
2. **HTTPS Server:** repeat the same operations using the HTTPS server and check if the username and password are still visible in the captured packets.

The practical work I followed can be summarized as:

```
Run server -> open localhost:8080 in Firefox -> enter credentials -> capture traffic  
on Loopback -> compare HTTP and HTTPS packet visibility
```

For the deliverable, I needed to answer these points for both HTTP and HTTPS:

1. the type of packets observed in Wireshark;
2. whether the username and password typed in the browser can be found among the packets;

3. where they can be found, or why they cannot be found.

4. Lab Environment and Tools

4.1 Working Environment

I performed the experiment inside the **Crownlabs Lab 2 virtual machine**. I used Terminal for running the server scripts, Firefox for opening the login page, and Wireshark for checking what was really sent on the loopback interface.

Tool	Role in the exercise
Terminal	Used to start the HTTP and HTTPS Python servers.
Firefox	Used to open the web login page and submit credentials.
Wireshark	Used to capture and inspect local web traffic.
Loopback interface	Used because the browser and server communicated through <code>localhost</code> .

4.2 Wireshark Capture Interface

For both parts of the Web exercise, I captured on Loopback: `lo`. This was the correct choice because I was using `localhost`. The browser and the server were both inside the same VM, so the traffic did not go outside through `enp1s0`.

The two URLs used in the experiment were:

```
http://localhost:8080
https://localhost:8080
```

4.3 Files Used in the Experiment

In the home directory of the VM, I found the files that were needed for this exercise:

```
test_server.py
test_server_https.py
cert.pem
key.pem
test_server/
```

I used `test_server.py` for the HTTP test and `test_server_https.py` for the HTTPS test. The files `cert.pem` and `key.pem` were related to the HTTPS certificate and private key.

5. Credentials Used for the Test

I used the same username and password in both tests, because this made the comparison easier and clearer.

Field	Value used
Username	mehran
Password	test123
Server port	8080
Server address	127.0.0.1 / localhost

Using the same values helped me check the result quickly. In the HTTP capture, I could clearly see these values in the packet body. In the HTTPS capture, I could not see them as readable strings in Wireshark.

6. HTTP Server Experiment

6.1 Server Execution

For the HTTP part, I first started the plain HTTP server with this command:

```
python3 test_server.py
```

The terminal showed that the server was listening on 127.0.0.1:8080:

```
Serving HTTP Server @ 127.0.0.1:8080...
```

After that, I opened this address in Firefox:

```
http://localhost:8080
```

After I pressed the login button, the server terminal confirmed that it received the credentials:

```
Login tentative for user: mehran, with password: test123
127.0.0.1 - - [15/May/2026 17:31:31] "POST /login HTTP/1.1" 200 -
```

6.2 Packets Observed in Wireshark

After the login, I stopped the capture and used the Wireshark display filter `http`. This made the HTTP packets easy to see. The important packets from my capture were:

No.	Source	Destination	Information
38619	127.0.0.1	127.0.0.1	GET / HTTP/1.1
38626	127.0.0.1	127.0.0.1	HTTP/1.0 200 OK (text/html)
38677	127.0.0.1	127.0.0.1	GET /favicon.ico HTTP/1.1
38684	127.0.0.1	127.0.0.1	HTTP/1.0 404 File not found
39651	127.0.0.1	127.0.0.1	POST /login HTTP/1.1 (text/plain)

For me, the most important packet was packet **39651**, because this was the POST /login request that was generated by the login button.

Wireshark - HTTP capture on Loopback: lo						
File Edit View Go Capture Analyze Statistics Tools Help						
http						
No.	Time	Source	Destination	Protocol	Length	Info
38619	743.965231	127.0.0.1	127.0.0.1	HTTP	512	GET / HTTP/1.1
38626	744.231421	127.0.0.1	127.0.0.1	HTTP	2607	HTTP/1.0 200 OK (text/html)
38677	744.486533	127.0.0.1	127.0.0.1	HTTP	512	GET /favicon.ico HTTP/1.1
38684	744.546238	127.0.0.1	127.0.0.1	HTTP	535	HTTP/1.0 404 File not found (text/html)
39651	769.089256	127.0.0.1	127.0.0.1	HTTP	562	POST /login HTTP/1.1 (text/plain)

Frame 39651: 562 bytes on wire (562 bytes captured on interface lo)
Ethernet II, Src: 00:00:00_00:00:00, Dst: 00:00:00_00:00:00
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 58476, Dst Port: 8080, Len: 496
Hypertext Transfer Protocol
POST /login HTTP/1.1
Host: localhost:8080
Content-Type: text/plain; charset=UTF-8
Content-Length: 33
Full request URI: http://localhost:8080/login
Line-based text data: text/plain (1 lines)
{ "user": "mehran", "pwd": "test123" }

Packet bytes / decoded data preview: the request body is visible in clear text.

Packets: 49221 - Displayed: 5 (0.0%) - Dropped: 0

Figure 1: HTTP capture filtered with `http`. The `POST /login` packet contains the clear-text request body with the username and password.

6.3 Username and Password Visibility in HTTP

When I opened the details of the HTTP packet, Wireshark showed the request body under `Line-based text data: text/plain`. I could read the payload directly:

```
{"user": "mehran", "pwd": "test123"}
```

This means that in the HTTP capture I found both the username and the password inside the packet itself. The reason is simple: HTTP does not encrypt the data. The browser sent the login information as readable text inside the `POST /login` request.

6.4 Interpretation of the HTTP Result

This result showed me why plain HTTP is not safe for login pages. In the browser, the password field was hidden with dots, but that only hides it on the screen. In the packet capture, the password was still sent as readable text because there was no encryption protecting the HTTP request body.

At the packet level, the structure was:

```
TCP connection -> HTTP POST /login -> clear-text payload containing user and pwd
```

So, the HTTP part demonstrated that application data can be visible in Wireshark when the communication is not encrypted.

7. HTTPS Server Experiment

7.1 Server Execution

For the HTTPS part, I stopped the HTTP server and then started the HTTPS server with this command:

```
python3 test_server_https.py
```

Then I opened the following URL in Firefox:

```
https://localhost:8080
```

Firefox displayed a security warning because the HTTPS server used a self-signed local certificate. This was expected in this lab. I selected the advanced option and accepted the risk so I could continue to the local HTTPS page.

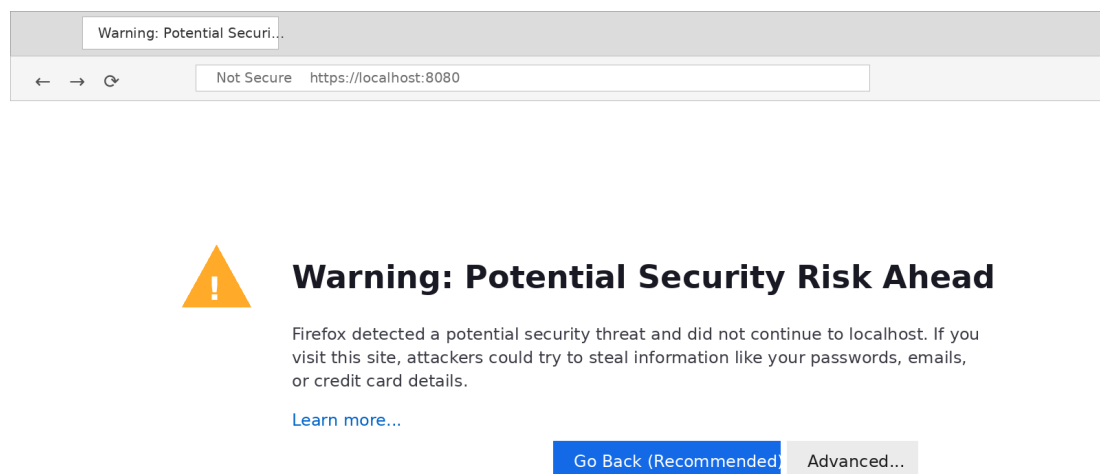


Figure 2: Firefox warning shown when opening the local HTTPS server with a self-signed certificate. I accepted the risk and continued, as required for the lab test.

After the login page opened, I entered the same values as before:

```
Username: mehran  
Password: test123
```

The server terminal showed that the login request arrived. I clicked the login button several times, so the terminal printed repeated lines. However, one successful `POST /login` was enough to prove that the HTTPS request was received:

```
Login tentative for user: mehran, with password: test123  
127.0.0.1 - - [15/May/2026 17:47:31] "POST /login HTTP/1.1" 200 -
```

The important point is that the server can print the username and password only because it decrypts the HTTPS request. This does not mean that Wireshark can read the same values from the network packets.

7.2 Packets Observed in Wireshark

In the HTTPS capture, I first used the display filter:

```
tcp.port == 8080
```

This showed packets between 127.0.0.1 and 127.0.0.1 on port 8080. I saw TCP connection packets and larger payload packets such as PSH, ACK. Since the HTTPS server was running on port 8080 instead of the normal HTTPS port 443, Wireshark mainly displayed the protocol as TCP.

Wireshark - HTTPS capture filtered with tcp.port == 8080						
File Edit View Go Capture Analyze Statistics Tools Help						
tcp.port == 8080						
No.	Time	Source	Destination	Protocol	Length	Info
7015	122.767891	127.0.0.1	127.0.0.1	TCP	74	36090 -> 8080 [SYN] Seq=0 Win=65495 Len=0
7016	122.767904	127.0.0.1	127.0.0.1	TCP	74	8080 -> 36090 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0
7017	122.767918	127.0.0.1	127.0.0.1	TCP	66	36090 -> 8080 [ACK] Seq=1 Ack=1 Win=65536 Len=0
7018	122.769416	127.0.0.1	127.0.0.1	TCP	1956	36090 -> 8080 [PSH, ACK] Len=1890
7019	122.769422	127.0.0.1	127.0.0.1	TCP	66	8080 -> 36090 [ACK] Seq=1 Ack=1891 Win=65536 Len=0
7023	122.782662	127.0.0.1	127.0.0.1	TCP	1956	8080 -> 36100 [PSH, ACK] Len=1890
7030	122.820209	127.0.0.1	127.0.0.1	TCP	2315	8080 -> 36090 [PSH, ACK] Len=2249
Frame 7015: 74 bytes on wire (588 bits) captured on interface lo (0 bits) Ethernet II, Src: 00:00:00:00:00:00, Dst: 00:00:00:00:00:00 Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 Transmission Control Protocol, Src Port: 36090, Dst Port: 8080, Seq: 0, Len: 0 The capture is on port 8080. The traffic is HTTPS/TLS, so the HTTP request body is encrypted. No clear text line such as {"user":"mehran","pwd":"test123"} is visible in this packet view.						
Packet bytes / decoded data preview						
Packets: 14790 - Displayed: 758 (5.1%) - Dropped: 0						

Figure 3: HTTPS capture filtered with tcp.port == 8080. The traffic appears as TCP segments carrying encrypted HTTPS/TLS data, and the login body is not readable.

Some of the observed packets were:

No.	Source	Destination	Protocol	Information
7015	127.0.0.1	127.0.0.1	TCP	36090 -> 8080 [SYN]
7016	127.0.0.1	127.0.0.1	TCP	8080 -> 36090 [SYN, ACK]
7017	127.0.0.1	127.0.0.1	TCP	36090 -> 8080 [ACK]
7018	127.0.0.1	127.0.0.1	TCP	36090 -> 8080 [PSH, ACK] Len=1890
7030	127.0.0.1	127.0.0.1	TCP	8080 -> 36090 [PSH, ACK] Len=2249

These packets confirm that data was exchanged, but they do not show the HTTP request body in readable form.

7.3 HTTP Filter Result in the HTTPS Capture

After the HTTPS login, I applied the Wireshark display filter:

```
http
```

The result was **0 displayed packets**. This was very important for the exercise, because it showed that Wireshark could not display a readable HTTP POST /login request in the HTTPS capture.

Wireshark - HTTPS capture after applying http display filter							
File	Edit	View	Go	Capture	Analyze	Statistics	Tools Help
http							
No.	Time	Source	Destination	Protocol	Length	Info	

No packets displayed by the http filter

This shows that the HTTPS login traffic was not visible as plain HTTP in Wireshark.

Hypertext Transfer Protocol: Protocol	Packets: 14790 - Displayed: 0 (0.0%) - Dropped: 0
---------------------------------------	---

Figure 4: HTTPS capture after applying the `http` display filter. No HTTP packets were displayed, which confirms that the HTTP content was not visible in clear text.

7.4 Username and Password Visibility in HTTPS

In the HTTPS capture, I could not find the username `mehran` or the password `test123` in the packet payload. This was different from the HTTP capture, where Wireshark showed **Line-based text data**. In the HTTPS capture, I did not see the readable body:

```
{"user":"mehran","pwd":"test123"}
```

The reason is that HTTPS puts the HTTP request inside an encrypted TLS channel. Wireshark captured the bytes on the network, but those bytes were encrypted. The server terminal printed the credentials only after the HTTPS server decrypted the request.

The HTTPS packet-level structure was:

```
TCP connection -> TLS/HTTPS encrypted channel -> encrypted HTTP POST /login data
```

Therefore, the username and password were protected from direct reading in Wireshark.

8. HTTP and HTTPS Comparison

8.1 Main Difference

The main difference I observed between the two captures was the visibility of the application-layer data.

Aspect	HTTP Server	HTTPS Server
URL used	http://localhost:8080	https://localhost:8080
Server script	test_server.py	test_server_https.py
Browser warning	No certificate warning	Certificate warning appeared because of local self-signed certificate
Main packets visible	TCP and readable HTTP	TCP packets carrying encrypted HTTPS/TLS data
HTTP filter result	HTTP packets displayed	0 HTTP packets displayed
Login request visibility	POST /login visible	No readable POST /login payload visible
Username/password in Wireshark	Visible as clear text	Not visible as clear text
Reason	HTTP has no encryption	HTTPS encrypts the HTTP payload using TLS

8.2 Security Interpretation

This experiment made the reason for HTTPS very clear. In HTTP, the password field looks hidden in the browser, but the actual packet still contains the password as text. In HTTPS, the same login data is encrypted before it is sent, so Wireshark does not reveal the credentials.

This does not mean that the server cannot see the data. The server has to decrypt the request to process the login. The difference is that the packet capture does not show the credentials in readable form when HTTPS is used.

9. Direct Answers for the Lab Deliverable

9.1 4.3.1 HTTP Server

1. Briefly report the type of packets observed in Wireshark.

In the HTTP capture, I observed TCP packets and HTTP packets on the loopback interface. The visible HTTP packets included GET /, HTTP/1.0 200 OK, GET /favicon.ico, HTTP/1.0 404 File not found, and most importantly POST /login HTTP/1.1.

2. Could you find the username and password typed in the browser among the packets?

Yes. I could find both the username and the password in Wireshark.

3. If so, where? If not, why?

They were visible in the payload of the HTTP POST /login request. In the packet details, Wireshark showed the request body under **Line-based text data: text/plain**. The visible content was:

```
{"user":"mehran","pwd":"test123"}
```

This happened because HTTP sends the application data in clear text.

9.2 4.3.2 HTTPS Server

1. Briefly report the type of packets observed in Wireshark.

In the HTTPS capture, I observed TCP packets on port 8080 carrying the encrypted HTTP-S/TLS communication. Since the server used the non-standard HTTPS port 8080, Wireshark mainly showed the packets as TCP segments, including handshake packets and larger data packets such as PSH, ACK. When I applied the `http` display filter, Wireshark displayed 0 packets.

2. Could you find the username and password typed in the browser among the packets?

No. I could not find the username `mehran` or the password `test123` inside the HTTPS packet payload in Wireshark.

3. If so, where? If not, why?

The username and password were not visible because HTTPS encrypts the HTTP request body using TLS. The server terminal showed that the login was received, but that was after decryption by the HTTPS server. In the Wireshark capture, the credentials were inside encrypted application data, not readable plain HTTP text.

10. Final Summary

I completed the Web exercise for both HTTP and HTTPS. In the HTTP part, I ran `test_server.py`, opened `http://localhost:8080`, entered `mehran` and `test123`, and captured the traffic in Wireshark. The POST /login packet clearly contained the request body with the username and password in clear text.

In the HTTPS part, I ran `test_server_https.py`, opened `https://localhost:8080`, accepted the local certificate warning, entered the same credentials, and captured the traffic again. The server received the login, but Wireshark did not show the credentials in readable form. When I used the `http` display filter, no readable HTTP packets appeared in the HTTPS capture.

The final results are summarized below.

Item	HTTP result	HTTPS result
Server command	<code>python3 test_server.py</code>	<code>python3 test_server_https.py</code>
URL	<code>http://localhost:8080</code>	<code>https://localhost:8080</code>
Wireshark interface	Loopback: lo	Loopback: lo
Important display filter	<code>http</code>	<code>tcp.port == 8080, then http</code>
Login packet	POST /login HTTP/1.1 visible	No readable HTTP packet visible
Credentials visible in Wireshark	Yes	No
Reason	Clear-text HTTP pay- load	TLS encryption protects the HTTP payload

This confirms the main lesson I learned from the exercise: **HTTP can expose sensitive information, while HTTPS protects the same information by encrypting the application-layer payload.**

References

1. Lab 4 assignment PDF, *Music and multimedia streaming over the Internet*, Politecnico di Torino, Academic year 2025/2026.
2. Lab 4 deliverable template, Section 4.3, Web.
3. Wireshark packet captures and screenshots performed in the Crownlabs Lab 2 virtual machine.